

Module 10 — LangChain with Python

Automate the patterns you already know

🎯 Learning goals

- Build **LCEL chains**: prompt templates, the pipe operator, and **structured output**.
- Wrap functions as **LangChain tools** and run them with **create_agent** — plus middleware and memory.
- Construct a **RAG chain** with a retriever and ChromaDB, then make retrieval an agent tool.
- Know when to use LangChain and when to stay hand-rolled.

What is LangChain?

- A Python/JS **framework for building LLM-powered applications** — the most widely adopted in the ecosystem.
- Standardises interfaces between components so they compose together (prompt, model, parser, retriever, tools).
- Huge integration surface: 100+ model providers, 50+ vector stores, document loaders, memory systems.
- Every chain is automatically **streamable**, **batchable**, and **invocable** — no extra wiring.

What can you build with it?

- **Chains** — prompt to model to structured output in one line.
- **Tool agents** — LLM decides which functions to call, framework runs the loop.
- **RAG pipelines** — retrieval, context injection, and generation wired together.
- **Beyond this module**: memory, conversation management, document ingestion, multi-model routing, evaluation.

LangChain building blocks

- **PromptTemplate**: reusable prompts with variables.
- **ChatModel**: LLM wrapper (OpenAI, Anthropic, local models).
- **Structured output**: `model.with_structured_output(PydanticModel)` returns typed objects.
- **OutputParser**: turn raw text into a string or parsed JSON.

Pipes

- The pipe operator (|) chains steps: prompt → model → parser
- Each step is independently testable and swappable
- LCEL (LangChain Expression Language) chains are the core of LangChain

LCEL chain — prompt | model | parser

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import JsonOutputParser

prompt = ChatPromptTemplate.from_messages([
    ("system", "Classify crew reports as: navigation, engineering, "
               "science, medical, operations. "
               "Return JSON: {{category, summary, priority}}"),
    ("human", "{report}"),
])

model = ChatOpenAI(model=MODEL, reasoning_effort="none", use_responses_api=True)

json_parser = JsonOutputParser()

chain = prompt | model | json_parser

result = chain.invoke({"report": "Plasma conduit 7-B ruptured"})
# {"category": "engineering", "summary": "...", "priority": "high"}
```

- → The pipe operator (|) chains steps: prompt → model → parser
- → Each step is independently testable and swappable

Structured output

```
from typing import Literal
from pydantic import BaseModel, Field

class Classification(BaseModel):
    category: Literal["navigation", "engineering", "science",
                      "medical", "operations"]
    summary: str = Field(description="One-sentence summary")
    priority: Literal["low", "medium", "high", "critical"]

chain = prompt | model.with_structured_output(Classification)

result = chain.invoke({"report": "Plasma conduit 7-B ruptured"})
# Classification(category='engineering', summary='...', priority='critical')
result.priority  # 'critical' — typed attribute access
```

→ The provider is constrained to the schema

🔌 How does the pipe (|) work?

- The `|` **looks** like a Unix pipe.
- LangChain uses operator overloading to define what `|` means for its own objects.
- Every chain step (prompt, model, parser) is a **Runnable**: anything with `.invoke()`, `.stream()`, and `.batch()`.
- Nothing runs until you call `**.invoke()**` —



Demo — Chains and prompts

Switch to terminal: python demo/01_chains_and_prompts.py



LangChain tools and agents

The tool-calling loop, automated

🛠 Giving tools to the agent

- The **@tool** decorator generates JSON Schema from type hints + docstring.
- `create_agent` builds the agent from model + tools + system prompt.
- It runs the loop: LLM → tool call → result → repeat (on LangGraph).
- `result["messages"]` holds every tool call and result — print it for a trace.

Tool-calling agent

```
from langchain_core.tools import tool
from langchain.agents import create_agent

@tool
def read_sensor(sensor_name: str) -> str:
    """Read the current value of a ship sensor."""
    data = SENSOR_DATA.get(sensor_name)
    if not data:
        return json.dumps({"error": f"Unknown sensor '{sensor_name}'"})
    return json.dumps(data)

@tool
def query_crew(department: str) -> str:
    """Look up crew members by department."""
    return json.dumps([c for c in CREW if c["department"] == department])

tools = [read_sensor, query_crew]
agent = create_agent(model, tools, system_prompt="You are the Pathfinder AI.")

result = agent.invoke({"messages": [{"role": "user", "content": "What is the shield integr
print(result["messages"][-1].text)
```

- → @tool decorator = schema from type hints + docstring
- → create_agent handles the loop — same pattern as Module 2

Middleware and memory

- **Middleware** hooks into the agent loop: before or after the model, or around each tool call.
- Use it for logging, call limits, human approval, PII redaction, and retries.
- **Memory** is a checkpointer: the agent saves its state after every step.
- Calls that share a **thread_id** continue the same conversation. A different id starts fresh.

Middleware and memory

```
from langchain.agents.middleware import ToolCallLimitMiddleware, wrap_tool_call
from langgraph.checkpoint.memory import InMemorySaver

@wrap_tool_call
def log_tool_calls(request, handler):
    print("→", request.tool_call["name"], request.tool_call["args"])
    return handler(request)

agent = create_agent(
    model, tools,
    middleware=[log_tool_calls, ToolCallLimitMiddleware(run_limit=4)],
    checkpointer=InMemorySaver(),
)
config = {"configurable": {"thread_id": "bridge"}}
agent.invoke({"messages": [{"role": "user", "content": "Shield integrity?"}]}, config)
agent.invoke({"messages": [{"role": "user", "content": "Above 90%?"}]}, config)
```

- → Middleware hooks the loop: logging, limits, human approval, PII, retries
- → A checkpointer + thread_id = conversation memory, no manual history list



Demo — LangChain agents

Switch to terminal: python demo/02_langchain_agents.py



RAG with LangChain

Retrieval chains in a few lines

🕮 LangChain RAG components

- **Retriever**: `.invoke(query)` returns documents. Wraps ChromaDB, FAISS, etc.
- **format_docs**: formats retrieved passages with source labels.
- **RunnablePassthrough**: passes the question through unchanged.
- **.assign(answer=...)**: adds the answer while keeping the docs — sources from one retrieval.

RAG chain with LCEL

```
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings
from langchain_core.runnables import RunnablePassthrough

vectorstore = Chroma.from_texts(texts, OpenAIEmbeddings(), metadatas=metas)
retriever = vectorstore.as_retriever(search_kwargs={"k": 5})

def format_docs(docs):
    return "\n\n".join(
        f"[Source {i+1}: {d.metadata['source']}] {d.page_content}"
        for i, d in enumerate(docs)
    )

answer_chain = (
    RunnablePassthrough.assign(context=lambda x: format_docs(x["docs"]))
    | prompt | model | StrOutputParser()
)
rag_chain = (
    {"docs": retriever, "question": RunnablePassthrough()}
    | RunnablePassthrough.assign(answer=answer_chain)
)
result = rag_chain.invoke("What caused the reactor anomaly?")
result["answer"], result["docs"]
```

- → Chroma.from_texts() — embed and store in one call
- → Same RAG pattern as Module 5, wired up with LCEL

Agentic RAG — retrieval as a tool

- A chain **always** retrieves once, with the user's exact words.
- Wrap the retriever in a `@tool` (`search_logs`) and give it to `create_agent`.
- The agent decides **whether** to search, **what** to search for, and whether to search again.
- It can combine logs with live tools — at the cost of more model calls.



Demo — LangChain RAG

Switch to terminal: python demo/03_langchain_rag.py



Serving chains as APIs

python module-10-langchain/demo/04_serving_chains.py

🔌 A chain is already an API

- Every **Runnable** has `ainvoke`, `abatch`, and `astream` — wrap them in **FastAPI** routes.
- Pydantic validates every request; OpenAPI docs are generated automatically.
- Older tutorials use **LangServe** `add_routes` — it is unmaintained and conflicts with current deps.
- For deploying **agents**, LangChain points to **LangGraph / LangSmith deployment**.

Chain to API in a few lines

```
from fastapi import FastAPI
from pydantic import BaseModel

chain = prompt | ChatOpenAI(model=MODEL, reasoning_effort="none", use_responses_api=True)

class ReportInput(BaseModel):
    report: str

class InvokeRequest(BaseModel):
    input: ReportInput

app = FastAPI(title="Report API")

@app.post("/classify/invoke")
async def classify_invoke(req: InvokeRequest):
    return {"output": await chain.ainvoke(req.input.model_dump())}
```

- → Same LCEL chain as demo 01 — ainvoke inside an async route
- → Run with: uvicorn server:app --reload

Endpoints in the demo

- **POST /classify/invoke** — body `{"input": {"report": "..."}}` → `{"output": {category, summary, priority}}`.
- **POST /classify/batch** — `chain.abatch(...)` over several reports.
- **POST /classify/stream** — `chain.astream(...)` + `StreamingResponse` (NDJSON of the partial result).
- Your FastAPI app can add ordinary routes too (e.g. `GET /health`).



Demo — Serving chains

Switch to terminal: python module-10-langchain/demo/04_serving_chains.py

The broader ecosystem

- **LangGraph** — stateful, graph-based multi-agent workflows (cycles, branching, human-in-the-loop).
- **LangSmith** — tracing and observability for debugging chains in production.



Putting it all together

Summary and exercises

⚖️ When to use a framework

- **Use LangChain** for standard patterns: chains, tool agents, RAG.
- **Skip it** when you need full control or minimal dependencies.
- You already know the patterns — the framework just accelerates them.
- Frameworks change fast — fundamentals endure.

Summary

LCEL is prompt, model, parser

Structured output comes from a schema, not a plea for JSON.

Agents and RAG are the same patterns

create_agent runs the tool loop. A retriever is the RAG chain, or a tool the agent calls.

Know the pattern, then pick the tool

LangChain is faster for chains, tool agents, and RAG — if you know what it does underneath.

Read the trace, and leave when it fights you

result["messages"] shows every call. If the framework gets in the way, go back to plain Python.

Exercises

01 — Chain basics: LCEL chain with structured output that classifies crew reports

02 — Tool agent: wraps the classifier + ship tools with create_agent (builds on 01)

03 — RAG chain: answer + sources chain, then agentic RAG with a search_logs tool (builds on 02)

04 — Chain API: expose the Horizon classifier as a FastAPI service



Module 10 — Complete

Next: edge topics